Semantic Analysis for a Context Free Grammar

Given a CFG, how to perform semantic analysis on the grammar rules is the theme for this document. Semantic analysis requires a certain context that is outlined below.

1. Depending upon the grammar G and its underlying language L(G), attributes of terminal and nonterminal symbols of G are defined.
2. A grammar symbol (terminal or nonterminal) may have one or more attributes associated with it. For example, the token, 129, may be defined using 3 attributes. Its value 129 is an attribute, its type which may be one from {short, int, long} is another attribute, the number of bytes required to save the value, {1,2, 4, 8} is another attribute.
3. The attributes of a token are initialized by the scanner when it is recognized. The attributes have to be communicated to the parser so that it can access the attribute values and operate on them in its rules.
4. The attributes of a nonterminal are defined by a parser. Semantic analyses are specified along with the grammar rules as semantic actions. These are performed by a parser along with parsing.
5. Semantic analysis requires nontrivial communication between the scanner and the parser. We have seen very restricted form of such communication in the assignment of Lab 5. A more general setting for communication is discussed here. We use an abstract grammar for the purpose of illustration.

Consider the grammar G given below with 2 nonterminals S and Y and S is the start symbol. There are 3 terminals { T1 T2 ,} in G with 4 rules.

S → S , Y | Y Y → T1 | T2

L(G) comprises of strings of the form : {T1 T2 T1 , T1 T1, T2 T2, T2 .....}.

The following attributes are defined for the grammar symbols.

Symbol	Attributes	Symbol	Attributes
T1	T1.fval : a real number T1.type : value 0 or 1	T2	T2.fval : a real number T2.type : value 0 or 1
Y	Y.fval : a real number Y.type : value 0 or 1	S	S.ival : an integer S.type : value 0 or 1 S.count : integer

In the theory of semantic analysis, we use the form, X.attr, to define an attribute named as "attr" and attach it to the symbol X, using dot (.).

It is the job of the scanner to initialize the attributes of the terminals. In our abstract grammar, we wish the scanner to assign the attributes to T1 and T2 as follows.

Symbol	Attributes	Symbol	Attributes
T1	T1.fval = 10.5 T1.type = 0	T2	T2.fval = 101 T2.type = 1

The parser has to assign attributes to nonterminals S and Y. For Y, its attributes are defined using the attributes in the rhs of its rules.

Symbol	Attributes	Symbol	Attributes
Y → T1	Y.fval = T1.fval Y.type = T1.type	Y → T2	Y.fval = T2.fval Y.type = T2.type

The parser has to assign attributes to nonterminals S as defined below.

Symbol	Attributes of S	Symbol	Attributes of S
S → Y	S.ival = Y.fval S.type = Y.type S.count = 1	S → S , Y	S.ival = Y.fval S.type = Y.type S.count = S.count+1

The set up for semantic analysis from the semantic analyser's point of view is complete. The remaining pieces of the puzzle is to establish all the required communication between the parser and its associated scanner. This part is explained using the norms of yacc.

- For a token the desired communication is achieved using the variable yylval. It is the user's responsibility to overload the definition of yylval in the yacc script.
- There are 2 different structures of yylval as defined above. The tokens T1, T2 and nonterminal Y require 2 attributes : a real number and a value 0 or 1. The nonterminal S require 3 attributes : a real number and a value 0 or 1 and an integer.
- The multiple forms of yylval is set up in yacc script, using the union and the struct features of C. Each different form of yylval is implemented as a different struct within the union as shown below.

```
%union
{
  struct X { float fval;  short type;  } attr2;
  struct Y { int ival; short type; int count;} attr3;
}
```

- The association of grammar symbols to its yylval is defined also in the yacc script as follows.

```
%token <attr2> T1 T2
%type <attr2> Y
%type <attr3> S
```

The lex script for this grammar follows and is also made available to you as a file, "lex-yacc-comm.l"

```
ws [ \t\n]+
letter [A-Za-z]
digit  [0-9]
id {letter}({letter}|{digit})*

%%
{ws}      {;/* No action */}
"T1"      {yylval.attr2.fval=10.5; yylval.attr2.type=0;  return T1;}
"T2"      {yylval.attr2.fval=101; yylval.attr2.type=1;  return T2;}
","       {return ',';}
```

The main contents of the yacc script is given in the following table and is also made available to you as a file "lex-yacc-comm.y".

```
%union
{struct X { float fval;  short type;  } attr2;
  struct Y { int ival; short type; int count;} attr3;
}
%token <attr2> T1 T2
%type <attr2> Y
%type <attr3> S
%%
```

```
S :   S ',' Y
{ $$.ival = $3.fval;$$.type = $3.type; ++$$.count;
  printf("ival = %d type = %d count = %d \n", $$.ival, $$.type,
          $$.count);}
 | Y
{ $$.ival = $1.fval;$$.type = $1.type; $$.count = 1;
  printf("ival = %d type = %d count = %d \n", $$.ival, $$.type,
          $$.count);}
Y :   T1
      {$$.fval = $1.fval;$$.type = $1.type;
```

	printf("fval = %f type = %d \n", $$.fval, $$.type);} | T2 {$$.fval = $1.fval;$$.type = $1.type; printf("fval = %f type = %d \n", $$.fval, $$.type);} ;

- You should generate the lexer and parser using the scripts given and observe its working on an input designed by you.

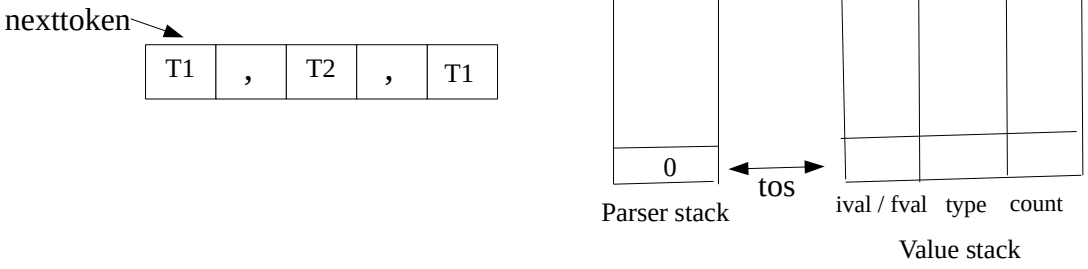
How Does Yacc Generated Parser Perform Semantic Analysis ?

Yacc performs semantic analysis during parsing. In the grammar of this document, semantic actions are added to the end of a rule in the rhs. Such a scheme is known syntax directed translation scheme (SDTS) and attributes used are known "synthesized attributes". There are other type of attributes that will be discussed in theory.

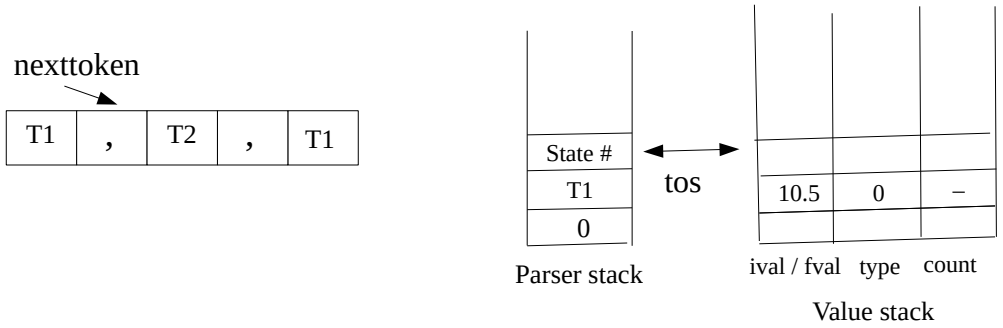
The answer to the posed question is that YACC uses 2 stacks. The parsing stack that contains grammar symbols and states you are familiar with. The other stack used by the parser is called a "value stack". The term "value" refers to semantic values (or attribute values) written by the semantic analysis designer. The value stack is exactly same as the parser stack in structure, except that in place of grammar symbols on the parser stack, their associated attribute values are kept on the value stack. The semantic actions are executed on the value stack just after a reduction takes place for a rule in the parsing stack.

The working of the parser in terms of the contents of the 2 stacks as the parser executes on an input is traced below. On the value stack, only the attribute values are shown.

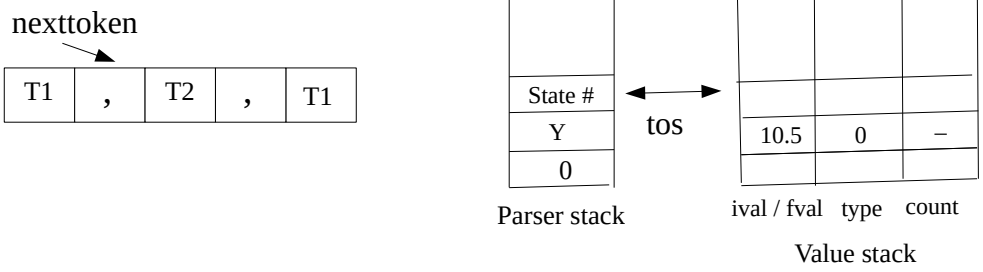
1. Initial Configuration :



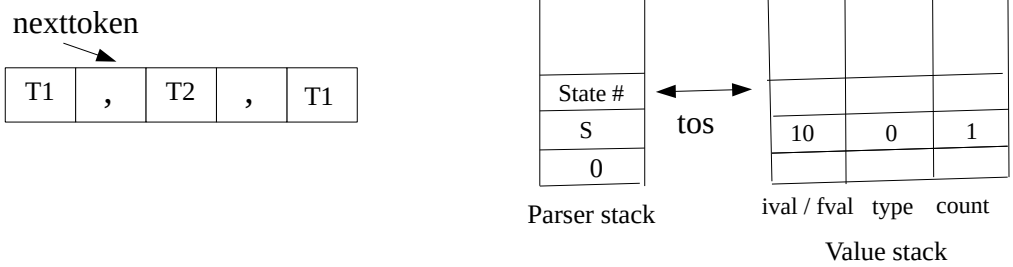
2. After reading token :



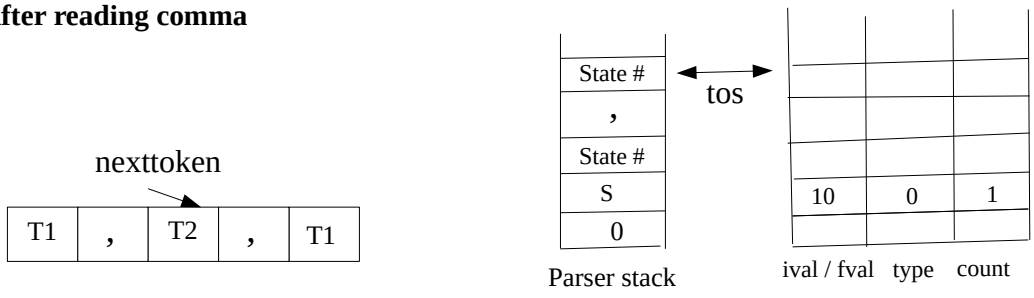
3. After reduction of T1 to Y



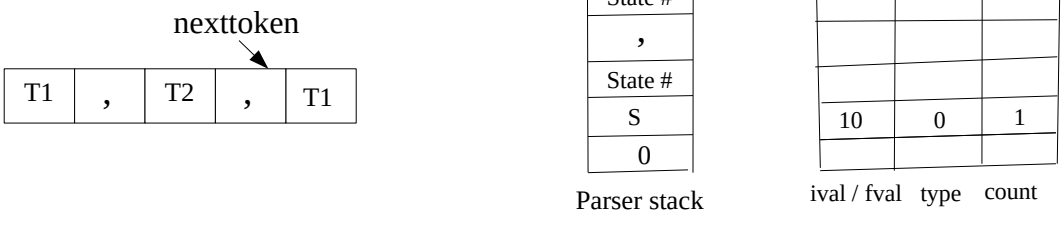
4. After reduction of Y to S



5. After reading comma



6. After reading T2



The rest of the parsing is left as an exercise for the reader. You are expected to understand the design choices of the semantic analysis writer through this exercise and then customize the semantic actions depending on the grammar rules and its their intended semantics.